

A few examples of how tar and rsync are used

Using the tar command, it's possible to create a single archive out of many files and directories. It maintains the directory staging structure and the same is applied while untarring the archive as well. Let's look at an example here, to understand both tar and rsync better.

Let's assume that there is a requirement to back up your Linux dev box on a periodic basis. The requirement includes backing up both the configuration and the user/system data as and when a change occurs. The directories to be backed up are `/boot`, `/etc`, `/home`, `/opt`, `/root`, `/usr` and `/var`.

Now, using tar, you can create a single stream or archive of all the directories and files to be backed up:

```
~] # tar cvfzP backup.tar.gz /boot/ /etc/ /home/ /opt/ /root/ /usr/ /var/
```

Where parameter 'c' helps to create the archive, 'v' to show what's going on to the end user (verbose mode), 'f' means to create an archive file instead of a device file, 'z' is to compress the archive file (using gzip) and 'P' is to maintain the absolute path.

Using rsync, you can copy this archive to a destination machine or folder:

```
~] # rsync -avz /root/backup.tar.gz root@10.10.3.228:/backup-test/
root@10.10.3.228's password:
building file list ... done
backup.tar.gz
```

```
sent 5834763 bytes received 42 bytes 1296623.33 bytes/sec
total size is 5832784 speedup is 1.00
~]#
```

However, if you want to copy the incremental changes alone to the destination machine, then rsync can be used alone. The first execution of rsync will transfer the entire contents to the destination and from there onwards it will copy the delta alone, comparing the files at the destination.

A simple example would be to copy the entire file system to the destination machine as shown below:

```
~] # rsync -avzR /boot/ /etc/ /home/ /opt/ /root/ /usr/ /var/
root@10.10.3.228:/backup-test/
root@10.10.3.228's password:
building file list...
```

The delta-transfer algorithm makes sure that it copies only the changed file from the source to destination from the next iteration onwards, thus reducing the overall size and increasing the speed of backup. Look through the man pages of rsync to understand more about the parameters used.

Automating the backup task

Linux has a beautiful utility called *crontab*, which can run the processes in the background at a specified time interval.

To perform the above mentioned automated task, we need to have a passwordless login to the destination machine. Refer to <http://www.thegeekstuff.com/2008/11/3-steps-to-perform-ssh-login-without-password-using-ssh-keygen-ssh-copy-id/>, explaining how to configure passwordless ssh.

A sample script, as shown below, can run the above task in a scripted approach. The script shown below doesn't use tar to create an archive of the entire file system for the earlier-mentioned requirement. Instead, it just uses rsync to copy the files (full files for the first attempt, and then transfers changes from there on).

```
#!/bin/sh
rsync -avzR /boot/ /etc/ /home/ /opt/ /root/ /usr/ /var/
root@10.10.3.228:/backup-test/
```

In the above snippet, 10.10.3.228 is the destination machine and *backup-test* is the destination folder in which the backup data needs to be stored.

If you need to run the above in an automated manner, just set up *crontab* as shown below:

```
00 00 * * * /usr/bin/backup.sh # assuming the above script is
named as backup.sh under /usr/bin
@thmMinute 12'o clock Everyday Every Month Every day of the week
```



By: Krishnaprasad K & Avinash Bendigeri

The author is a software engineers at Dell Inc. He can be reached at krishnaprasad_k@dell.com

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

Your favourite magazine on
Open Source is now on the Web, too.

LinuxForU.com

Follow us on Twitter@LinuxForYou

